

Employee Payroll Calculator — Project Documentation

1. Overview

The Employee Payroll Calculator is a payroll management solution originally developed in **VBA.NET** as a standalone desktop application in **Visual Studio**, following **object-oriented programming (OOP)** practices and a custom WinForms-based user interface (UI).

It has since been **ported to Python**, using **Gradio** for the web interface, **SQLite** for persistence, and deployed on **Hugging Face Spaces** for public demonstration.

Key goals: - Accurate payroll computations with support for overtime, pre-tax and post-tax deductions, and taxes. - Maintainable, modular architecture through separation of concerns. - Seamless user experience both in the original desktop and new web-based versions. - Cross-platform and cloud accessibility in the Python port.

2. Project Evolution Timeline

Phase 1 – VBA.NET (Visual Studio Desktop App) - Built as a WinForms application with multiple forms for data entry, calculations, and reporting. - Classes for **Employee**, **PayrollRun**, **DeductionCalculator**, **TaxCalculator**, and **DatabaseManager**. - Data persisted locally in Access DB or CSV files.

Phase 2 – Python Web App (Gradio + SQLite) - Recreated the OOP structure using Python modules and classes. - Moved UI to browser with Gradio's input/output components. - Migrated data storage to SQLite for relational storage and querying. - Deployed to Hugging Face Spaces for live, interactive demos.

3. Features

- **Gross/Net Pay Calculation** – hourly, salary, overtime, bonuses.
 - **Deductions** – pre-tax (401k, HSA, FSA) and post-tax (benefits, garnishments).
 - **Tax Computations** – Federal, State, Social Security, Medicare.
 - **History & Reporting** – store, filter, and export payroll records.
 - **Cross-platform UI** – desktop WinForms (original) → browser-based Gradio (port).
-

4. Before/After Architecture

VBA.NET Architecture: - **UI Layer:** WinForms-based data entry & reports. - **Domain Layer:** Business logic classes in separate modules. - **Data Layer:** Access/CSV storage.

Python Architecture: - **UI Layer:** Gradio application (`app.py`). - **Domain Layer:** `calculator.py`, `config.py`, `models.py`. - **Data Layer:** SQLite storage (`db.py`).

5. Data Model (SQLite)

Tables - **employees** - ID, employee_code, name, department. - **payroll_runs** - ID, FK to employee, period dates, hours, rates, gross, deductions, net. - **deduction_lines** - FK to payroll_runs, deduction code, amount, pretax flag.

6. Configuration (config.py)

```
FED_WITHHOLDING_RATE = 0.12
STATE_WITHHOLDING_RATE = 0.05
SOC_SEC_RATE = 0.062
MEDICARE_RATE = 0.0145
ENABLE_STATE_TAX = True
ENABLE_SOC_SEC = True
ENABLE_MEDICARE = True
```

7. Payroll Calculation Flow

1. **User inputs** hours, rate, deductions, period dates.
 2. **Gross Pay** calculated (regular + overtime × multiplier).
 3. **Pre-tax deductions** subtracted.
 4. **Taxes** computed (enabled tax types).
 5. **Post-tax deductions** subtracted.
 6. **Net Pay** displayed and saved to DB.
-

8. Python Functions

```
def calculate(inputs, cfg) -> dict:
    """Return gross, pretax_total, taxes, net_pay, breakdown."""

def init_db(db_path="payroll.db"):
    pass

def insert_payroll_run(run_dict):
    pass

def query_history(employee_code=None, start=None, end=None):
    pass
```

9. Running Locally

```
pip install -r requirements.txt
python app.py
```

10. Deployment (Hugging Face)

- Create new Space → SDK: **Gradio**.
 - Push `app.py`, `payload/` package, `requirements.txt`.
 - Deploy → app launches in browser.
-

11. Usage Guide

1. Open app.
 2. Enter employee details and pay period.
 3. Input hours, rates, deductions.
 4. Click **Calculate**.
 5. Save record.
 6. Filter & export history.
-

12. Security Notes

- Do not store PII beyond necessary employee code.
 - Anonymize before public demo.
-

13. Migration Notes (VBA.NET → Python)

- Ported class structure 1:1 where possible.
 - Converted event-driven form logic to Gradio callbacks.
 - Replaced file-based storage with relational SQLite.
-

14. Appendix

Example CSV Export:

```
employee_code,period_start,period_end,hours_regular,hours_overtime,rate_hourly,gross,pretax,taxes,posttax
E001,2025-07-01,2025-07-15,80,5,25,2187.50,150.00,275.38,50.00,1712.12
```